

^From:.\+@[\\w.]\+

```
re.compile([\\w.]\+)
```

Decoding the Regex Engine

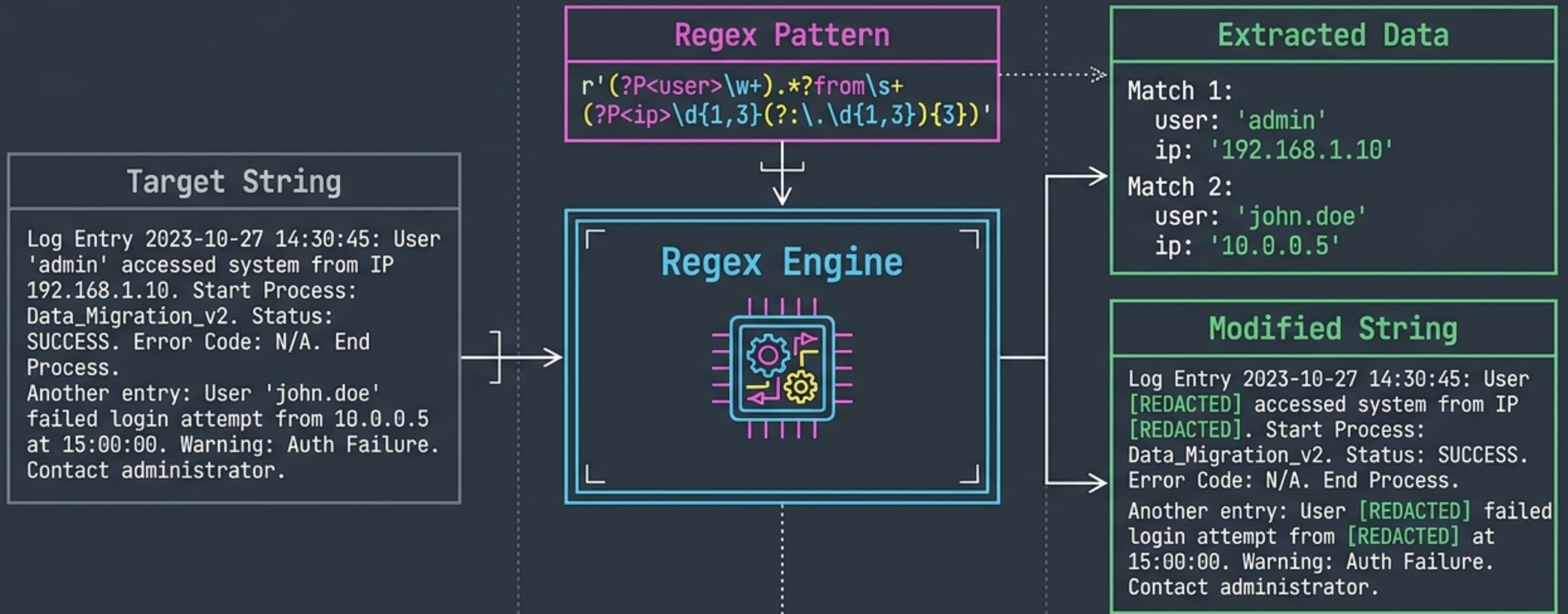
A visual blueprint for text pattern matching
and data extraction in Python.

```
def re.compile([a-zA-Z0-9]\+, ?):  
    strings = '[\\w.]\+'  
    match.group()
```

^([\\.|'|[a-zA-Z0-9]\+)

```
re.compile('(202[0-3]), {')  
match = new string  
match.group()  
...
```


The Engine and the Pattern



A regular expression does not itself perform any action — it is merely a pattern.

The **Python re module** supplies the **Engine** (to **search**, **extract**, or **replace**).
The **Regex syntax** supplies the **Pattern** (the rules).

Imperative Search vs. Pattern Search

	<code>str.find()</code>	<code>re.search()</code>
Approach	Literal string matching only.	Pattern-based matching.
Capability	Finds exact character sequences (e.g., <code>'From: '</code>).	Finds logical sequences (e.g., <code>'From: '</code> followed by an <code>email address</code>).
Scaling	Requires dozens of lines of if/else loops for complex queries.	Handled in a single, expressive pattern line.

Traditional string methods tell the computer **how** to search step-by-step. Regular expressions declare **what** to find.

The Python re Module Toolkit

Search

`re.match(pattern, string)`

Checks for a match only at the beginning of the string. If the string does not begin with the pattern, returns None.

Search

`re.search(pattern, string)`

Searches the entire string for the first occurrence of the pattern anywhere within it. Returns a match object if found.

Extract

`re.findall(pattern, string)`

Returns a list of all non-overlapping matches of the pattern in the string. Most commonly used for extraction.

Extract

`re.finditer(pattern, string)`

Returns an iterator of match objects (retains position info).

Modify

`re.sub(pattern, repl, string)`

Replaces pattern occurrences with a new string.

Modify

`re.split(pattern, string)`

Splits string at each pattern occurrence into a list.

Meta-characters: The Regex Alphabet (Part 1)

Anchors (Position matters)

- **^** : Start of string -> **^From** matches lines beginning with 'From'.
- **\$** : End of string -> **@gmail.com\$** matches strings ending with **@gmail.com**.

Character Shortcuts (Notice the case inversion)

- **\d** vs **\D** : Decimal digit (0-9) vs NON-digit. (**\d\d\d** matches a 3-digit sequence).
- **\w** vs **\W** : Word character (letters, digits, _) vs NON-word character.
- **\s** vs **\S** : Whitespace (space, tab, newline) vs NON-whitespace.

Meta-characters: The Regex Alphabet (Part 2)

The Wildcard

`.` : Matches ANY single character (except newline). -> '`h.t`' matches '`hat`', '`hot`', '`hit`'.

Character Classes (Custom limits)

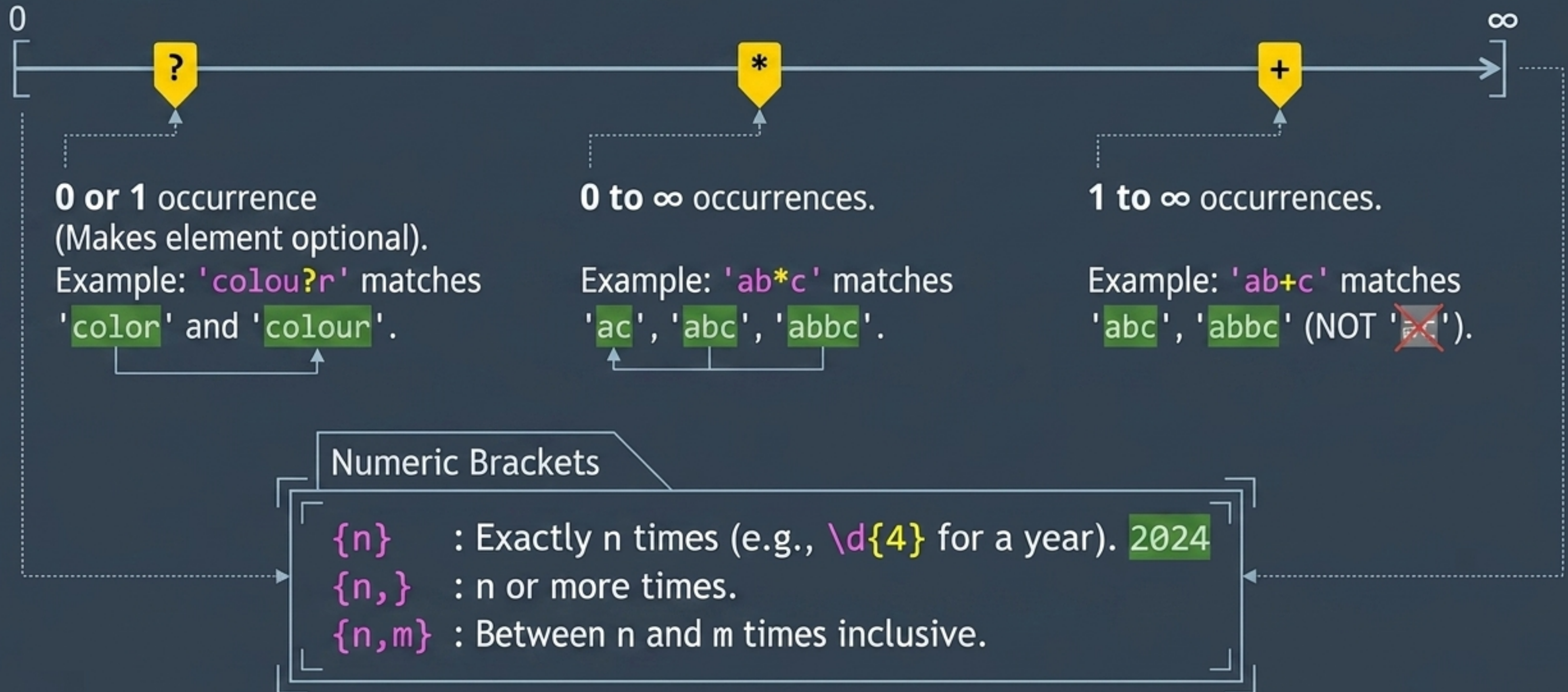
`[abc]` : Matches any one character inside the brackets. -> '`[aeiou]`' matches any vowel.

`[^abc]` : Negated class. Matches any character except those in brackets. -> '`[^0-9]`' matches any non-digit.

Capturing Groups

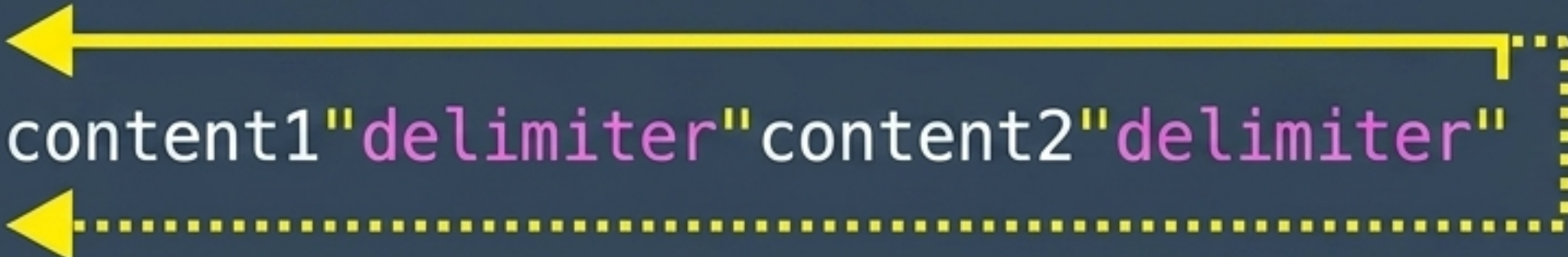
`(abc)` : Matches the pattern and captures it as a distinct group for extraction. -> '`(\d+)`' isolates a sequence of digits.

Quantifiers: The Multiplier Scale



Mechanical Rules: Greedy vs. Lazy

Greedy



content1"delimiter"content2"delimiter"

Concept: By default, quantifiers (**span***, **+**) are **greedy**—they consume as many characters as possible before attempting to match the rest of the pattern.

Lazy



content1"delimiter"content2"delimiter"

The Fix: Append **?** to a quantifier (e.g., **<span-*?>** or **+?**) to make it lazy. It will match as few characters as possible.

Use Case: Critical when extracting specific content between HTML tags or quotation marks.

Synthesis: Deconstructing the Pattern

Anchor to the exact start of the line.

^From: .+@[\w.]+

Literal text match.

One or more of ANY character
(The sender's name).

Literal symbol match.

Custom Class. One or more word characters OR dots
(The domain name).

Find lines starting with 'From:' and capture the full email address that follows.

Putting it together: re.search()

```
1 import re
2 hand = open('mbox-short.txt')
3 for line in hand:
4     line = line.rstrip() # Clean trailing whitespace
5     match = re.search('^From: .+@[\\w.]+', line)
6     if match:
7         print(line)
```

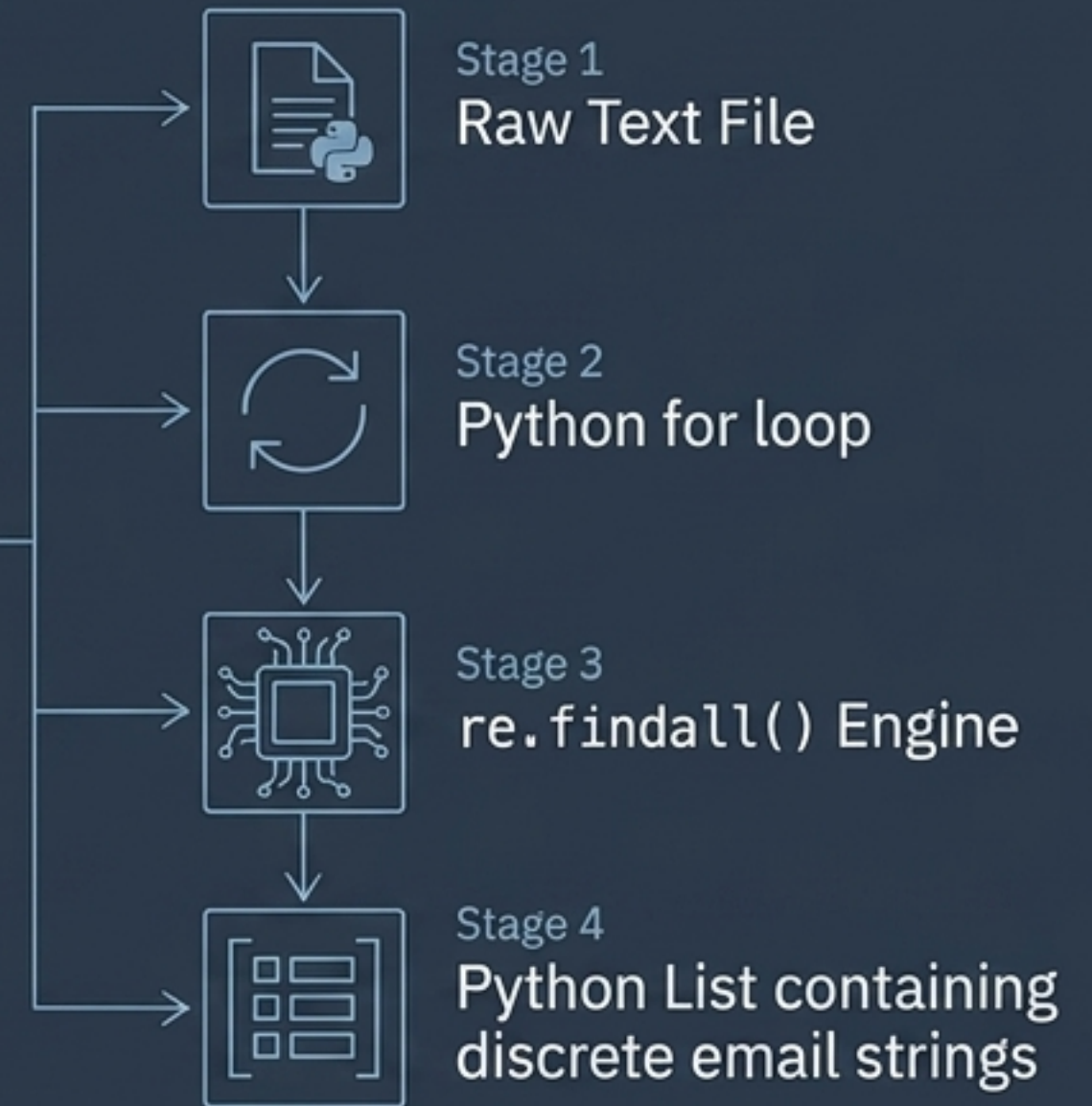
..... import re must be called first.

..... The loop feeds the stripped string into the engine one line at a time.

..... re.search evaluates True if the pattern is found, ignoring non-matching lines seamlessly.

Data Extraction Flow: re.findall()

```
1 import re
2 hand = open('mbox-short.txt')
3 emails = list()
4 for line in hand:
5     line = line.rstrip()
6     found = re.findall('[\w.]+@[.\w.]+', line)
7     if found:
8         emails = emails + found
```



Unlike `re.search` which just confirms a match exists, `re.findall` automatically extracts the matching characters and returns them as a Python list, perfect for **aggregating** datasets.

The Regex Blueprint Reference

Functions	Meta-Characters	Quantifiers
match	Anchors	<code>*</code> , <code>+</code> , <code>?</code> , <code>{n}</code>
search	Wildcards	
findall	Shortcuts <code>\d</code> , <code>\w</code> , <code>\s</code>	
sub		

Regex is not magic. It is a strict, highly logical grammar.
Master the alphabet, combine with the engine, and extract anything.